

AI Software Supply-Chain Security and Vulnerability Detection

Abstract

Artificial intelligence has changed the structure of software supply chains in two directions at once. First, AI is increasingly used within software assurance workflows for vulnerability detection, prioritization, triage, and repair. Second, AI systems themselves have become complex supply-chain artifacts composed of code, package dependencies, container images, model weights, datasets, prompts, adapters, evaluation assets, and external services. These two developments create a coupled research problem: how to secure the AI-enabled software supply chain, and how to use AI to detect vulnerabilities and malicious manipulations across that supply chain. This survey synthesizes the literature on classical software supply-chain attacks, learning-based vulnerability detection, malicious package detection, dependency-risk analysis, model and dataset supply-chain security, provenance and software bills of materials, and recent large-language-model-based detection and repair techniques. We argue that the field is converging on a layered assurance stack that combines artifact provenance, structural program analysis, behavioral and graph representations, cross-artifact correlation, and human-in-the-loop verification. At the same time, the literature repeatedly exposes the same bottlenecks: noisy labels, weak ground truth, limited cross-project generalization, benchmark leakage, ecosystem drift, opaque package behaviors, unsafe model serialization formats, and the absence of mature AI-specific provenance standards. The survey organizes the area into method families, identifies unifying abstractions, contrasts strengths and limitations, and outlines research prospects for trustworthy end-to-end assurance of modern AI software supply chains. The recent literature also makes clear that model hubs and model-loading pipelines now play a role analogous to repositories and package managers in traditional software supply chains, extending the attack surface from source and binaries to learned artifacts and deserialization logic. ¹

Introduction

Software supply-chain security studies how compromise propagates through the artifacts and processes by which software is built, packaged, distributed, updated, and reused. In the traditional open-source setting, this includes package managers, mirrors, build systems, release channels, dependency graphs, and update metadata; in the AI setting, the same logic extends to model hubs, serialized checkpoints, fine-tuned adapters, data pipelines, evaluation corpora, and MLOps orchestration. The literature now treats these as a continuous attack surface rather than two disjoint domains. Foundational work on package-manager attacks and secure update frameworks established that trust in distribution metadata, maintainers, mirrors, and clients is fragile unless backed by explicit provenance and cryptographic verification [1–9]. More recent work shows that the same reasoning applies to pretrained models, where hubs, loaders, and serialization formats act as the “package managers” of machine learning [85–97, 110]. ²

A central reason this topic now deserves a unified survey is that AI appears on both sides of the security equation. On one side, machine learning and, more recently, foundation models have been applied to source-code vulnerability detection, clone-based vulnerability discovery, dependency-risk analysis, malware-package screening, and automated repair [34–84]. On the other side, AI artifacts have

themselves become supply-chain liabilities: pretrained models can embed backdoors or hidden payloads, loaders can trigger arbitrary code execution, datasets can be poisoned, and fine-tuning or model-reuse workflows can silently import malicious behavior [85–110]. The result is a feedback loop in which AI is simultaneously a security tool, an attack surface, and a supply-chain dependency. ³

The software-engineering literature has already documented that open-source reuse improves productivity while also importing security debt through transitive dependencies and vulnerable code reuse [23–33, 64–72]. The package-ecosystem literature then sharpened this picture by showing that malicious updates, typosquatting, dependency confusion, and ecosystem-specific social engineering all exploit structural properties of modern registries and resolver semantics [10–22]. In parallel, the vulnerability-detection literature moved from hand-built signatures, code property graphs, and patch-differencing toward deep learning over token, sequence, graph, and hybrid representations [34–63]. The AI-supply-chain literature extends these lines of work by showing that model repositories and loaders replicate many of the same weaknesses, but with higher opacity and weaker operational norms around verification [85–97]. ⁴

This survey adopts a deliberately broad but technically coherent scope. It includes: secure update and provenance frameworks; malicious-package and dependency-vulnerability detection in open-source ecosystems; AI-assisted software vulnerability detection and repair; model, dataset, and MLOps supply-chain attacks; and provenance, SBOM, and AI bill-of-material concepts that support end-to-end assurance. The survey does **not** attempt to cover the full adversarial-machine-learning universe; instead, it focuses on those attacks and defenses that matter directly for software and AI supply-chain integrity, artifact verification, and vulnerability discovery. Recent surveys of automated vulnerability detection characterize the maturing but still unstable empirical landscape of learning-based techniques, while newer position and system papers argue that machine learning models now require supply-chain thinking comparable to that long applied to code and packages [58–63, 84–97, 110]. ⁵

From this literature, four recurring themes emerge. First, provenance is the backbone of supply-chain defense, but provenance remains incomplete or weakly enforced in both software and AI artifacts [2–4, 29–33, 92–97]. Second, structural reasoning still matters: graph-based representations, dependency trees, code property graphs, and behavior graphs remain among the most effective abstractions for both vulnerability detection and malicious-package analysis [16, 19, 26, 39, 44, 48, 49, 57, 69]. Third, data quality and evaluation realism remain unresolved obstacles, especially for AI-based vulnerability detection [58–63]. Fourth, “safe” AI model sharing is not yet mature: recent work shows that model-loading security claims often fail under adversarially crafted artifacts [85–90]. ⁶

Methods

Survey Design and Scope

This survey is organized around method families rather than venues or application silos. That choice reflects the fact that very similar technical ideas recur across seemingly different problems. For example, provenance graphs appear in secure updates, SBOM analysis, MLOps lineage, and dependency-risk tracing; behavioral modeling appears in malicious npm detection, package-update differential analysis, and model-loading exploit studies; and graph neural networks appear in source-code vulnerability detection, binary- or behavior-level security analysis, and package-behavior classification [2–9, 16–22, 39–57, 85–97]. The survey therefore groups the literature into six interacting directions: provenance and trust frameworks; package-ecosystem and dependency security; code-level vulnerability detection; AI-augmented repair and DevSecOps; AI artifact and model supply-chain security; and cross-cutting evaluation concerns. ⁷

Methodologically, this survey draws on peer-reviewed conference papers, journal articles, workshop papers, and arXiv preprints with nontrivial technical content. The emphasis is on research contributions that define a threat model, methodology, benchmark, or design framework, rather than practitioner commentary. This is particularly important for AI supply-chain security, where academic literature remains newer and more fragmented than the traditional software-supply-chain literature. Recent academic work on secure MLOps, model provenance, model-loading security, and AI bills of materials shows that the field is moving rapidly but has not yet stabilized around shared standards or benchmark conventions [92–97]. ⁸

Provenance, Trust, and Artifact Integrity

The first method family concerns preventive control rather than detection in the narrow sense. Secure update frameworks such as TUF-related work, in-toto, Diplomat, Mercury, and later Artemis formalize trust delegation, threshold signatures, repository compromise resistance, and supply-chain-step attestation [1–4, 9]. Their main contribution is conceptual as much as cryptographic: they decompose software distribution into explicitly signed roles and verifiable steps. In the software-supply-chain literature, this line of work underpins the idea that vulnerability detection alone is insufficient; one also needs authenticated lineage and non-repudiable build metadata. ⁹

Recent SBOM work extends provenance from “who signed this release?” to “what components and subcomponents are inside it?” Stakeholder studies and tool landscape analyses show that SBOM adoption is growing, but coverage, resolution fidelity, and transitive completeness remain inconsistent [29–33]. Java-focused studies further document that real-world SBOM generation is difficult because dependency scopes, build plugins, vendored code, and ecosystem-specific metadata do not map cleanly onto a single inventory abstraction [30]. This becomes even harder for AI stacks, where a materially complete inventory must describe not just packages and images, but also model checkpoints, tokenizer versions, dataset slices, fine-tuning corpora, adapters, and evaluation prompts [92–97]. ¹⁰

The AI-specific provenance literature is still early-stage, but it already reveals an important shift: model artifacts need lifecycle-level, not file-level, transparency. Atlas and related work argue that verifiable AI provenance requires tracking contributions and integrity across data collection, training, fine-tuning, evaluation, packaging, and deployment [92, 94, 95]. The proposed AI bill-of-materials literature makes a similar argument, but focuses on inventory and assurance semantics. At a minimum, these works imply that future AI supply-chain scanners should reason over composed provenance graphs rather than isolated files or repository snapshots [93, 96, 97]. ¹¹

Package Ecosystems, Dependency Graphs, and Malicious Package Detection

The second method family treats supply-chain security as an ecosystem-analysis problem. The package-manager literature began by studying mirrors, update channels, and registry trust, but the threat model soon expanded to include resolver abuse, namespace confusion, malicious publish events, and stealthy update-stage payloads [1, 5, 7, 10, 11]. Reviews and SoK papers consistently note that interpreted-language ecosystems such as npm and PyPI are especially exposed because installation hooks, dynamically fetched payloads, and lightweight publisher friction make it easy to weaponize packages at scale [5, 7, 10, 11]. ¹²

Empirical studies contributed the next step by moving from conceptual taxonomies to ecosystem measurement. MALOSS-style designs measured supply-chain attacks on interpreted-language package managers [11]; repository-mining work used source repositories and package metadata to identify suspicious behavior [13]; and later studies systematically analyzed malicious PyPI and npm packages in the wild [14–22]. The field then diversified into several detection strategies. Some systems emphasize

static behavior modeling and attack-vector extraction [12, 19, 21]; others rely on code clustering, semantic matching, graph-based behavior models, or differential analysis across package versions [17, 19, 20]. The core technical pattern is to model installation-time or update-time behavior instead of just package text. This is crucial because many malicious packages are small, short-lived, and sparsely labeled, making pure lexical or metadata-based learning brittle [12, 16–22]. ¹³

Dependency-vulnerability work complements malicious-package detection by focusing on non-malicious but vulnerable packages. Studies of npm applications show that vulnerability exposure is heavily shaped by transitive dependencies and adoption lag [24–27]. Dependency trees are not merely bookkeeping structures; they are propagation channels that determine whether vulnerabilities remain reachable, whether a fix can be adopted without breakage, and where remediation responsibility lies in the tree [26]. This motivated work on vulnerability propagation analysis, remediation bots, and impact assessment [26–28]. A critical lesson from this line is that package-security decisions cannot be made from a flat package list: path-sensitive dependency reachability and ecosystem version dynamics matter. ¹⁴

A further methodological insight is that software-package detection is increasingly converging with provenance and SBOM analysis. Version-differential scanners, policy-driven SBOM studies, and attack-path inference over logical graphs all point toward the same end state: detection engines should combine artifact identity, transitive structure, behavioral deltas, and risk propagation paths, rather than treating “vulnerable dependency,” “malicious update,” and “untrusted provenance” as separate problems [20, 28–33]. That same integration will be necessary in AI ecosystems, where package dependencies, model files, and external inference services interact within a single deployment chain. ¹⁵

Code-Level Vulnerability Detection with Learning and Graph Representations

The third method family addresses vulnerability detection in source code and related program representations. Classic static analysis and patch-based matching remain essential, but the last decade produced a large literature on learned detectors built over tokens, ASTs, code slices, data/control-flow graphs, and code property graphs [34–72]. Early sequence-based systems such as VulDeePecker and SySeVR introduced the idea of extracting vulnerability-centric code slices or “code gadgets” that preserve relevant context while remaining learnable [35, 36, 38]. Subsequent work increased granularity using token-level localization, hybrid representations, and fine-grained detectors [37, 42, 54, 55]. ¹⁶

Graph representations became especially influential because vulnerabilities are often relational phenomena rather than simple lexical patterns. Devign, DeepWukong, behavior-graph models, graph simplification methods, and related approaches represent code as a graph whose nodes and edges encode syntax, control flow, data flow, or higher-level semantic relations [39, 44, 48, 49, 57, 69]. Their promise is twofold: they preserve nonlocal interactions such as pointer aliasing and taint propagation, and they allow message-passing networks to learn structural cues that token models miss. However, the literature also shows that graph sophistication does not automatically translate into robust generalization. As benchmarks became more realistic, many reported gains shrank, suggesting that some earlier improvements were entangled with dataset artifacts rather than genuine semantic learning [58–63]. ¹⁷

Another major branch focuses on clone- or patch-oriented vulnerability discovery. VUDDY, CENTRIS, MVP, V1SCAN, and related systems detect reused vulnerable code in large code bases by combining clone detection, patch signatures, or binary/source classification [64–68]. These approaches are important for supply-chain security because many real vulnerabilities enter software through reuse rather than being newly authored in the target repository. Compared with end-to-end neural detectors, patch- and reuse-oriented methods are often more interpretable and operationally useful, particularly for identifying 1-day vulnerabilities in vendored or modified third-party components [64–68]. ¹⁸

Despite methodological variety, the literature increasingly converges on three empirical facts. First, representation quality matters more than model novelty when vulnerability labels are noisy. Second, cross-project generalization remains weak unless datasets control for duplicates, function clones, and near-miss patch leakage. Third, realistic deployment requires triage and explanation support, because developers must distinguish true vulnerabilities from model-induced false alarms [58–63]. Recent survey papers, data-quality critiques, and sampling studies all reinforce this point, and false-alarm studies show that human factors remain the primary bottleneck in practical adoption [58–63]. ¹⁹

AI-Assisted Vulnerability Repair and DevSecOps

The fourth method family concerns the use of pretrained models and LLMs for downstream security workflows. Vulnerability repair systems such as VulRepair show that sequence-to-sequence pretrained models can learn recurring security-fix patterns, especially when repair is framed as conditional code generation [73]. Follow-up work expands this line to end-to-end pipelines that predict, classify, localize, and attempt to repair vulnerabilities or at least suggest patches [81, 82]. At a systems level, the broader DevSecOps literature positions these tools as copilots embedded in CI/CD and secure review pipelines rather than one-shot detectors [84]. ²⁰

The LLM-for-security literature has grown quickly, but the evidence is nuanced. Short empirical studies using ChatGPT and similar models found nontrivial utility for vulnerability classification and patch suggestion, but also large variability, hallucinations, and weak consistency [74]. More recent methods attempt to address this by grounding LLMs with structured code context, retrieval augmentation, or code property graphs. SAFE, Vul-RAG, LLMxCPG, and related work exemplify this direction: they add syntactic and semantic grounding, curated security knowledge, or repository-level context to reduce spurious judgments [75–80]. These systems represent a shift from “LLM as a free-form classifier” toward “LLM as a constrained reasoner over structured security evidence.” ²¹

Benchmarking work is especially important here. Recent ACL-scale evaluation of LLMs and LLM-based agents on practical repository vulnerability detection suggests that strong general-purpose models can be competitive on some tasks, but performance degrades when the benchmark requires repository navigation, patch reasoning, or subtle reachability judgments [77]. This aligns with earlier lessons from learned vulnerability detection: benchmark realism, context completeness, and label fidelity are more decisive than headline model scale. The state of the art is thus best understood as hybridization: LLMs are increasingly effective when paired with program analysis, retrieval, and task decomposition, but they do not obviate static or graph-based methods [73–84]. ²²

AI Artifacts, Model Sharing, and Model-Supply-Chain Security

The fifth method family focuses on the supply chain of AI artifacts themselves. Empirical studies of pretrained model repositories show that model hubs resemble software registries in both utility and risk. They centralize discovery and reuse, but they also consolidate trust and amplify compromise if artifact validation is weak [85, 86]. Position papers now explicitly frame model weights, checkpoints, and sharing hubs as supply-chain artifacts that require lineage, attestation, and policy, rather than informal “research assets” [96, 110]. ²³

The technical attack surface is broader than backdoors alone. One line of work studies hidden malicious payloads embedded in model parameters, as in EvilModel, EvilModel 2.0, and MalModel [88–90]. Another examines model-reuse attacks, where importing a pretrained component reveals a latent adversarial capability [100]. A third examines traditional backdoors in the supply-chain frame: BadNets, Trojaning Attack on Neural Networks, Hidden Trigger Backdoor Attacks, Neural Cleanse, Fine-Pruning, and Spectral Signatures all show that pretrained or third-party models can carry malicious input-triggered behavior

that survives downstream integration [98–105]. In supply-chain terms, these works show that model acquisition is not just “using a dependency” ; it is inheriting latent decision logic that may be malicious, fragile, or poorly understood. ²⁴

The newest and arguably most operationally urgent line studies unsafe model loading. Recent work on the security of loading machine learning models demonstrates that popular frameworks and hubs often rely on incomplete or misleading safe-loading narratives. Even ostensibly security-oriented modes may be bypassed, and data-based formats are not inherently secure when full model reconstruction still permits dangerous imports or gadget execution [87]. This matters because the model-loading step is the AI analogue of package installation: it is where untrusted artifacts meet privileged execution environments. The literature now strongly suggests that model-security evaluation should include loader semantics, serialization formats, hub scanners, and user security perception—not just training-time threats [85–87, 91–97]. ²⁵

The MLOps literature generalizes this one step further. Secure-MLOps surveys argue that attacks occur throughout collection, training, packaging, deployment, and monitoring, and that ATLAS-style lifecycle taxonomies are more appropriate than one-off threat analyses [97]. This perspective is valuable because it connects software supply-chain controls with AI-specific risks such as dataset poisoning, adapter trojans, and insecure inference components. The emerging research agenda is therefore end-to-end: provenance, scanning, policy, secure loaders, and downstream runtime telemetry must be treated as one assurance pipeline [92–97]. ²⁶

Challenges and Prospects

The most persistent challenge across the entire literature is ground truth. In code vulnerability detection, labels often come from commit heuristics, synthetic suites, partial CVE mappings, or benchmark curation choices that leak artifacts into both train and test splits [41–43, 58–63]. In malicious-package detection, labels are often sparse, short-lived, or post hoc, while benign packages frequently contain installation scripts or obfuscation-like behavior that looks suspicious out of context [12, 14–22]. In AI model supply chains, there is even less mature labeling infrastructure: malicious or vulnerable models may be discovered only after public release, and there is no universal repository-scale equivalent of the CVE/CWE ecosystem [85–97]. This means that much of the literature still optimizes on incomplete proxies rather than stable security truth. ²⁷

A second challenge is representation mismatch. Supply-chain attacks are inherently multi-artifact and multi-stage, but many detectors still operate on just one view: source tokens, package metadata, a dependency tree snapshot, or a serialized model file. This encourages blind spots. For example, a malicious npm package may appear innocuous in source form but reveal its intent through installation hooks and version-to-version deltas; a model file may appear benign in archive structure but become dangerous when interpreted by a specific loader; and a vulnerable dependency may be irrelevant if it is unreachable at runtime. Future systems therefore need cross-artifact reasoning that joins code, metadata, dependency structure, build provenance, and execution context into a shared graph [20, 26–33, 85–97]. ²⁸

A third challenge is generalization under ecosystem drift. Programming languages, package ecosystems, registry policies, model serialization formats, and LLM prompting conventions all change quickly. Systems trained on yesterday’ s malicious-package families, yesterday’ s vulnerability corpora, or yesterday’ s model-loading rules may degrade silently when attacker behavior shifts. The literature on data sampling, benchmark quality, and practical deployment repeatedly shows that cross-project and cross-time robustness is weaker than in-distribution metrics suggest [58–63, 77]. This suggests that

evaluation should move toward temporally split benchmarks, ecosystem-transfer tests, and policy-aware continuous updating rather than static leaderboard comparisons. ²⁹

A fourth challenge is that the field lacks a mature AI-native provenance standard. SBOM work in software already struggles with granularity, transitivity, and tool interoperability [29–33]. AI amplifies this problem because meaningful provenance must describe dataset derivation, model ancestry, fine-tuning lineage, tokenizer and embedding dependencies, serialization choices, banned or deprecated loaders, and sometimes even prompt templates and tool bindings [92–97]. The prospect here is promising: Atlas, AIBOM proposals, and shared-model transparency frameworks all point toward richer lifecycle capture. But without agreement on schemas, trust semantics, and attestation boundaries, detection systems will continue to lack the input data needed for reliable AI supply-chain assurance. ³⁰

The most promising technical prospect is a layered assurance architecture. At the base layer, cryptographic provenance and lifecycle attestation verify origin and transformations [2–4, 9, 92–97]. At the analysis layer, static analysis, differential analysis, and graph reasoning characterize artifact behavior [20, 26, 39, 44, 48, 49, 57, 69]. At the learning layer, domain-specific models and LLMs rank, summarize, and localize likely issues [46–57, 73–84]. At the operational layer, remediation bots and policy engines use dependency context and developer feedback to prioritize action [27–33, 84]. In this view, AI does not replace classical supply-chain security; it becomes a higher-level inference engine built on provenance and program semantics. ³¹

Another promising direction is to unify vulnerability detection and supply-chain risk analysis. Today these are often separate communities: one studies vulnerable code, the other malicious or vulnerable artifacts in ecosystems. Yet many practical incidents involve both. A reused third-party component may contain a latent vulnerability; a malicious package may also exploit a loader vulnerability; a model artifact may embed a backdoor and trigger arbitrary code execution on load. Future research should therefore treat “artifact risk” as a joint function of intrinsic vulnerability, malicious intent, transitive exposure, and provenance uncertainty. The literature already contains the ingredients for this unification, but not yet a dominant framework for combining them [23–33, 64–72, 85–110]. ³²

Conclusion

AI software supply-chain security and vulnerability detection is no longer a niche intersection of software engineering and adversarial machine learning. It is a core systems-security problem created by the convergence of open-source reuse, automated DevSecOps, model sharing, and AI-enabled code intelligence. The literature shows that traditional supply-chain insights remain valid: provenance, delegation, authenticated metadata, and explicit trust boundaries are foundational [1–9]. It also shows that modern detection demands richer representations: graph structures, behavioral deltas, dependency propagation paths, and repository-level context frequently outperform purely local views [16–22, 26, 39, 44, 48, 49, 57, 69]. Finally, the newest AI-specific studies show that models themselves must now be treated as software supply-chain artifacts, with all the attendant needs for safe loading, scanning, provenance, and compromise recovery [85–97, 110]. ³³

The strongest overall conclusion from the surveyed literature is that no single methodological family is sufficient. Cryptographic provenance without behavioral analysis misses malicious semantics; learning-based detection without trustworthy labels invites brittle overfitting; secure loading without lifecycle provenance still leaves opportunities for poisoned training inputs or malicious adapters; and LLM-based triage without structural grounding remains error-prone. The field’s most plausible path forward is therefore compositional: provenance-first, graph-aware, benchmark-rigorous, and human-auditable systems that treat code, packages, models, data, and deployment logic as one integrated supply chain. If

that integration succeeds, AI may become not merely another attack surface, but a powerful component of end-to-end software supply-chain assurance. ³⁴

References

1. Justin Cappos, Justin Samuel, Scott M. Baker, and John H. Hartman. “A Look in the Mirror: Attacks on Package Managers.” ACM CCS, 2008.
2. Santiago Torres-Arias, Hammad Afzali, Trishank K. Kuppusamy, Reza Curtmola, and Justin Cappos. “in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes.” USENIX Security, 2019.
3. Trishank K. Kuppusamy, Santiago Torres-Arias, Vladimir Diaz, and Justin Cappos. “Diplomat: Using Delegations to Protect Community Repositories.” USENIX NSDI, 2016.
4. Trishank K. Kuppusamy, Vladimir Diaz, and Justin Cappos. “Mercury: Bandwidth-Effective Prevention of Rollback Attacks Against Community Repositories.” USENIX ATC, 2017.
5. Marc Ohm, Henrik Plate, Andreas Sykosch, and Michael Meier. “Backstabber’s Knife Collection: A Review of Open Source Software Supply Chain Attacks.” DIMVA, 2020.
6. William Enck and Laurie Williams. “The Top Five Challenges in Software Supply Chain Security.” IEEE Security & Privacy, 2022.
7. Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. “SoK: Taxonomy of Attacks on Open-Source Software Supply Chains.” IEEE Symposium on Security and Privacy, 2023.
8. Marc Ohm and Daniel Stucke. “SoK: Practical Detection of Software Supply Chain Attacks.” ARES, 2023.
9. Marina Moore, Trishank K. Kuppusamy, and Justin Cappos. “Artemis: Defanging Software Supply Chain Attacks in Multi-repository Update Systems.” ACSAC, 2023.
10. Henrik Bos. “A Review of Attacks Against Language-Based Package Managers.” arXiv preprint, 2023. arXiv:2302.08959.
11. Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. “Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages.” NDSS, 2021.
12. Adriana Sejfia and Max Schäfer. “Practical Automated Detection of Malicious npm Packages.” ICSE, 2022. arXiv:2202.13953.
13. Linh Vu, Julian Cohen, Matthew Hyatt, et al. “Towards Using Source Code Repositories to Identify Software Supply Chain Attacks.” CCS, 2020.
14. Wenbo Guo, Zhengzi Xu, Chengwei Liu, Cheng Huang, Yong Fang, and Yang Liu. “An Empirical Study of Malicious Code in PyPI Ecosystem.” ASE, 2023.

15. Ning Li, Kai Chen, et al. “MalWuKong: Towards Fast, Accurate, and Multilingual Detection of Malicious Code Poisoning in OSS Supply Chains.” ASE, 2023.
16. Piergiorgio Ladisa, Serena Elisa Ponta, Nicola Ronzoni, Matias Martinez, and Olivier Barais. “On the Feasibility of Cross-Language Detection of Malicious Packages in npm and PyPI.” ACSAC, 2023. arXiv:2310.09571.
17. Wentao Liang, Xiang Ling, Jingzheng Wu, Tianyue Luo, and Yanjun Wu. “A Needle Is an Outlier in a Haystack: Hunting Malicious PyPI Packages with Code Clustering.” ASE, 2023.
18. Cheng Huang, Nannan Wang, Ziyang Wang, Siqi Sun, Lingzi Li, Junren Chen, Qianchong Zhao, Jiaxuan Han, Zhen Yang, and Lei Shi. “DONAPI: Malicious NPM Packages Detector Using Behavior Sequence Knowledge Mapping.” arXiv preprint, 2024. arXiv:2403.08334.
19. Yiheng Huang, Ruisi Wang, Wen Zheng, Zhuotong Zhou, Susheng Wu, Shulin Ke, Bihuan Chen, Shan Gao, and Xin Peng. “SpiderScan: Practical Detection of Malicious NPM Packages Based on Graph-Based Behavior Modeling and Matching.” ASE, 2024.
20. Fabian Niklas Froh, Matías Federico Gobbi, and Johannes Kinder. “Differential Static Analysis for Detecting Malicious Updates to Open Source Packages.” SCORED@CCS, 2023.
21. Yu Zhang, Haipeng Qu, Lingyun Ying, and Linghui Wang. “Maldet: An Automated Malicious npm Package Detector Based on Behavior Characteristics and Attack Vectors.” TrustCom, 2024.
22. Wenbo Guo, Zhongwen Chen, Zhengzi Xu, Chengwei Liu, Ming Kang, Shiwen Song, Chengyue Liu, Yijia Xu, Weisong Sun, and Yang Liu. “Understanding NPM Malicious Package Detection: A Benchmark-Driven Empirical Analysis.” arXiv preprint, 2026. arXiv:2603.27549.
23. Antonios Gkortzis, Daniel Feitosa, and Diomidis Spinellis. “Software Reuse Cuts Both Ways: An Empirical Analysis of Its Relationship with Security Vulnerabilities.” Journal of Systems and Software, 2021.
24. Mahmoud Alfadel, Diego Elias Costa, Mouafak Mkhallalati, Emad Shihab, and Bram Adams. “On the Threat of npm Vulnerable Dependencies in Node.js Applications.” arXiv preprint, 2020. arXiv:2009.09019.
25. Bodin Chinthanet, Raula G. Kula, Shane McIntosh, Takashi Ishio, Akinori Ihara, and Kenichi Matsumoto. “Lags in the Release, Adoption, and Propagation of npm Vulnerability Fixes.” Empirical Software Engineering, 2021.
26. Chengwei Liu, Sen Chen, Lingling Fan, Bihuan Chen, Yang Liu, and Xin Peng. “Demystifying the Vulnerability Propagation and Its Evolution via Dependency Trees in the NPM Ecosystem.” ICSE, 2022. arXiv:2201.03981.
27. Hamid Mohayjeji, Andrei Agaronian, Eleni Constantinou, Nicola Zannone, and Alexander Serebrenik. “Investigating the Resolution of Vulnerable Dependencies with Dependabot Security Updates.” MSR, 2023.
28. Bonan Ruan, Zhiwei Lin, Jiahao Liu, Chuqi Zhang, Kaihang Ji, and Zhenkai Liang. “Propagation-Based Vulnerability Impact Assessment for Software Supply Chains.” ASE, 2025.

29. Trevor Stalnaker, Nathan Wintersgill, Oscar Chaparro, Massimiliano Di Penta, Daniel M. Germán, and Denys Poshyvanyk. “BOMs Away! Inside the Minds of Stakeholders: A Comprehensive Study of Bills of Materials for Software Systems.” ICSE, 2024.
30. Musard Balliu, Benoit Baudry, Sofia Bobadilla, Mathias Ekstedt, Martin Monperrus, Javier Ron, Aman Sharma, Gabriel Skoglund, César Soto-Valero, and Martin Wittlinger. “Challenges of Producing Software Bill of Materials for Java.” IEEE Security & Privacy, 2023. arXiv:2303.11102.
31. Derek Garcia, Mehdi Tarrit Mirakhorli, Schuyler Dillon, Kevin Laporte, Matthew Morrison, Henry Lu, Viktoria Koscinski, Christopher Enoch, Mohamad Fazelnia, and Roger Chen. “A Landscape Study of Open-Source Tools for Software Bill of Materials and Supply Chain Security.” SVM@ICSE, 2025.
32. Luis Soeiro, Thomas Robert, and Stefano Zacchiroli. “Wild SBOMs: A Large-Scale Dataset of Software Bills of Materials from Public Code.” MSR, 2025. arXiv:2503.15021.
33. Oleksii Novikov, Davide Fucci, Oleksandr Adamov, and Daniel Méndez. “Policy-Driven Software Bill of Materials on GitHub: An Empirical Study.” PROFES, 2025.
34. Rebecca L. Russell, Louis Y. Kim, Lei H. Hamilton, Tomo Lazovich, Jacob Harer, Onur Ozdemir, Paul M. Ellingwood, and Marc W. McConley. “Automated Vulnerability Detection in Source Code Using Deep Representation Learning.” ICMLA, 2018.
35. Deqing Zou, Shouhuai Xu, Zhen Li, Wei Hua, et al. “VulDeePecker: A Deep Learning-Based System for Vulnerability Detection.” NDSS, 2018.
36. Deqing Zou, Sujuan Wang, Shouhuai Xu, Zhen Li, and Hai Jin. “A Deep Learning-Based System for Multiclass Vulnerability Detection.” IEEE Transactions on Dependable and Secure Computing, 2019.
37. Zhen Li, Deqing Zou, Shouhuai Xu, Zhaoxuan Chen, Yawei Zhu, and Hai Jin. “VulDeeLocator: A Deep Learning-Based Fine-Grained Vulnerability Detector.” IEEE Transactions on Dependable and Secure Computing, 2021.
38. Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. “SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities.” IEEE Transactions on Dependable and Secure Computing, 2022. arXiv:1807.06756.
39. Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. “Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks.” NeurIPS, 2019.
40. Guozhu Lin, Wenbo Xiao, Jun Zhang, and Yang Xiang. “Deep Learning-Based Vulnerable Function Detection: A Benchmark.” ICICS, 2019.
41. Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. “A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries.” MSR, 2020.
42. Yizheng Zheng, Saurabh Pujar, B. Lewis, and Leonardo Buratti. “D2A: A Dataset Built for AI-Based Vulnerability Detection Methods Using Differential Analysis.” ICSE-SEIP, 2021.

43. Yizheng Chen, Zhoujie Ding, Lamya Alowain, Xinyun Chen, and David Wagner. "DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning-Based Vulnerability Detection." RAID, 2023.
44. Xinyu Cheng, Hongyu Wang, Jiguang Hua, Guang Xu, and Yulei Sui. "DeepWukong: Statically Detecting Software Vulnerabilities Using Deep Graph Neural Network." ACM Transactions on Software Engineering and Methodology, 2021.
45. Hong Wang, Guang Ye, Zongwei Tang, Shin Hwei Tan, and Zhendong Wang. "Combining Graph-Based Learning with Automated Data Collection for Code Vulnerability Detection." IEEE Transactions on Information Forensics and Security, 2020.
46. Hazim Hanif and Sergio Maffei. "VulBERTa: Simplified Source Code Pre-Training for Vulnerability Detection." IJCNN, 2022.
47. Michael Fu and Chakkrit Tantithamthavorn. "LineVul: A Transformer-Based Line-Level Vulnerability Prediction." MSR, 2022.
48. Bin Yuan, Yifan Lu, Yilin Fang, Yueming Wu, Deqing Zou, Zhen Li, Zhi Li, and Hai Jin. "Enhancing Deep Learning-Based Vulnerability Detection by Building Behavior Graph Model." ICSE, 2023.
49. Xin-Cheng Wen, Yupan Chen, Cuiyun Gao, Hongyu Zhang, Jie M. Zhang, and Qing Liao. "Vulnerability Detection with Graph Simplification and Enhanced Graph Representation Learning." ICSE, 2023.
50. Na Guo, Xin Li, Hao Yin, Yao Gao, Jinxiu Zhou, Xiang Luo, Qingsong Shen, and Zhilong Xu. "VulHunter: An Automated Vulnerability Detection System Based on Deep Learning and Bytecode." ICICS, 2020.
51. Seyed Mohsen Ghaffarian and Hamid R. Shahriari. "Neural Software Vulnerability Analysis Using Rich Intermediate Graph Representations of Programs." Information and Software Technology, 2021.
52. Wei Niu, Xiaoyan Zhang, Xianbin Du, Lei Zhao, Rui Cao, and Mohsen Guizani. "A Deep Learning-Based Static Taint Analysis Approach for IoT Software Vulnerability Location." Measurement, 2020.
53. Yuelong Wu, Jintian Lu, Yunyi Zhang, and Shuyuan Jin. "Vulnerability Detection in C/C++ Source Code With Graph Representation Learning." CCWC, 2021.
54. Md Nadeem Islam and Chanchal K. Roy. "Towards Token-Level Bug Detection in Source Code." ESEC/FSE, 2018.
55. Yonghua Liu et al. "Token-Level Deep Learning Model for Source Code Vulnerability Detection." Information, 2020.
56. Peng Zeng, Guanjin Lin, Jun Zhang, and Ying Zhang. "Intelligent Detection of Vulnerable Functions in Software Through Neural Embedding-Based Code Analysis." International Journal of Network Management, 2023.

57. Muhammad Fakhrur Rozi, Tao Ban, Seiichi Ozawa, Akira Yamada, Takeshi Takahashi, and Daisuke Inoue. "Securing Code With Context: Enhancing Vulnerability Detection Through Contextualized Graph Representations." IEEE Access, 2024.
58. Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. "Deep Learning Based Vulnerability Detection: Are We There Yet?" IEEE Transactions on Software Engineering, 2022. arXiv:2009.07235.
59. Nima Shiri Harzevili et al. "A Survey on Automated Software Vulnerability Detection Using Machine Learning and Deep Learning." arXiv preprint, 2023. arXiv:2306.11673.
60. Md Nizam Uddin, Yihe Zhang, and Xiali Hei. "Deep Learning Aided Software Vulnerability Detection: A Survey." arXiv preprint, 2025. arXiv:2503.04002.
61. Xu Yang, Shaowei Wang, Yi Li, and Shaohua Wang. "Does Data Sampling Improve Deep Learning-Based Vulnerability Detection? Yeas! and Nays!" ICSE, 2023.
62. Yuejun Guo and Seifeddine Bettaieb. "Data Quality Issues in Vulnerability Detection Datasets." arXiv preprint, 2024. arXiv:2410.06030.
63. Hong Jin Kang, Khai Loong Aw, and David Lo. "Detecting False Alarms from Automatic Static Analysis Tools: How Far Are We?" ICSE, 2022.
64. Seulbae Kim, Seunghoon Woo, Heejo Lee, and Hakjoo Oh. "VUDDY: A Scalable Approach for Vulnerable Code Clone Discovery." IEEE Symposium on Security and Privacy, 2017.
65. Seunghoon Woo, Sunghan Park, Seulbae Kim, Heejo Lee, and Hakjoo Oh. "CENTRIS: A Precise and Scalable Approach for Identifying Modified Open-Source Software Reuse." ICSE, 2021.
66. Yang Xiao, Bihuan Chen, Chendong Yu, Zhengzi Xu, Zimu Yuan, Feng Li, Binghong Liu, Yang Liu, Wei Huo, Wei Zou, and Wenchang Shi. "MVP: Detecting Vulnerabilities Using Patch-Enhanced Vulnerability Signatures." USENIX Security, 2020.
67. Seunghoon Woo, Eunjin Choi, Heejo Lee, and Hakjoo Oh. "V1SCAN: Discovering 1-Day Vulnerabilities in Reused C/C++ Open-Source Software Components Using Code Classification Techniques." USENIX Security, 2023.
68. Mansour Ahmadi, Reza Mirzazade Farkhani, Ryan Williams, and Long Lu. "Finding Bugs Using Your Own Code: Detecting Functionally Similar yet Inconsistent Code." USENIX Security, 2021.
69. Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. "Modeling and Discovering Vulnerabilities with Code Property Graphs." IEEE Symposium on Security and Privacy, 2014.
70. Riccardo Scandariato, James Walden, Aram Hovsepyan, and Wouter Joosen. "Predicting Vulnerable Software Components via Text Mining." IEEE Transactions on Software Engineering, 2014.
71. Fabio Sabetta and Michele Bezzi. "A Practical Approach to the Automatic Classification of Security-Relevant Commits." ICSME, 2018.

72. Yaqin Zhou and Ankit Sharma. “Automated Identification of Security Issues from Commit Messages and Bug Reports.” ASE, 2017.
73. Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Van Nguyen, and Dinh Q. Phung. “VulRepair: A T5-Based Automated Software Vulnerability Repair.” ESEC/FSE, 2022.
74. Michael Fu, Chakkrit Tantithamthavorn, Van Nguyen, and Trung Le. “ChatGPT for Vulnerability Detection, Classification, and Repair: How Far Are We?” APSEC, 2023. arXiv:2310.09810.
75. Van Nguyen, Surya Nepal, Tingmin Wu, Xingliang Yuan, and Carsten Rudolph. “SAFE: Advancing Large Language Models in Leveraging Semantic and Syntactic Relationships for Software Vulnerability Detection.” arXiv preprint, 2024. arXiv:2409.00882.
76. Xueying Du, Geng Zheng, Kaixin Wang, Jiayi Feng, Wentai Deng, Mingwei Liu, Bihuan Chen, Xin Peng, Tao Ma, and Yiling Lou. “Vul-RAG: Enhancing LLM-Based Vulnerability Detection via Knowledge-Level RAG.” arXiv preprint, 2024. arXiv:2406.11147.
77. Alperen Yildiz, Sin G. Teo, Yiling Lou, Yebo Feng, Chong Wang, and Dinil Mon Divakaran. “Benchmarking LLMs and LLM-based Agents in Practical Vulnerability Detection for Code Repositories.” ACL, 2025. arXiv:2503.03586.
78. Yikun Li et al. “CleanVul: Automatic Function-Level Vulnerability Detection in Code Commits Using LLM Heuristics.” arXiv preprint, 2024. arXiv:2411.17274.
79. Xiaohu Du, Ming Wen, Jiahao Zhu, Zifan Xie, Bin Ji, Huijun Liu, Xuanhua Shi, and Hai Jin. “Generalization-Enhanced Code Vulnerability Detection via Multi-Task Instruction Fine-Tuning.” Findings of ACL, 2024.
80. Ahmed Lekssays, Hamza Mouhcine, Khang Tran, Ting Yu, and Issa Khalil. “LLMxCPG: Context-Aware Vulnerability Detection Through Code Property Graph-Guided Large Language Models.” USENIX Security, 2025.
81. Quanjun Zhang, Chunrong Fang, Bowen Yu, Weisong Sun, Tongke Zhang, and Zhenyu Chen. “Pre-Trained Model-Based Automated Software Vulnerability Repair: How Far Are We?” arXiv preprint, 2023. arXiv:2308.12533.
82. Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Yuki Kume, Van Nguyen, Dinh Phung, and John C. Grundy. “AIBugHunter: A Practical Tool for Predicting, Classifying and Repairing Software Vulnerabilities.” arXiv preprint, 2023. arXiv:2305.16615.
83. Yiannis Charalambous, Norbert Tihanyi, Ridhi Jain, Youcheng Sun, Mohamed Amine Ferrag, and Lucas Cordeiro. “A New Era in Software Security: Towards Self-Healing Software via Large Language Models and Formal Verification.” arXiv preprint, 2023. arXiv:2305.14752.
84. Michael Fu, Jirat Pasuksmit, and Chakkrit Tantithamthavorn. “AI for DevSecOps: A Landscape and Future Opportunities.” ACM Transactions on Software Engineering and Methodology, 2025. arXiv:2404.04839.

85. Wenxin Jiang, Nicholas Synovic, Rohan Sethi, Aryan Indarapu, Matt Hyatt, Taylor R. Schorlemmer, George K. Thiruvathukal, and James C. Davis. “An Empirical Study of Artifacts and Security Risks in the Pre-trained Model Supply Chain.” SCORED@CCS, 2022.
86. Beatrice Casey, Joanna C. S. Santos, and Mehdi Mirakhorli. “A Large-Scale Exploit Instrumentation Study of AI/ML Supply Chain Attacks in Hugging Face Models.” arXiv preprint, 2024. arXiv:2410.04490.
87. Gabriele Digregorio, Marco Di Gennaro, Stefano Zanero, Stefano Longari, and Michele Carminati. “On the (In)Security of Loading Machine Learning Models.” arXiv preprint, 2026. arXiv: 2509.06703.
88. Zhaowei Wang, Chengyuan Liu, and Xiaofeng Cui. “EvilModel: Hiding Malware Inside of Neural Network Models.” ISCC, 2021. arXiv:2107.08590.
89. Zhaowei Wang, Chengyuan Liu, and Xiaofeng Cui. “EvilModel 2.0: Bringing Neural Network Models into Malware Attacks.” arXiv preprint, 2021. arXiv:2109.04344.
90. Jiayi Hua, Kailong Wang, Min Wang, Guohao Bai, Xiaopeng Luo, and Haoyu Wang. “MalModel: Hiding Malicious Payload in Mobile Deep Learning Models with Black-Box Backdoor Attack.” arXiv preprint, 2024. arXiv:2401.02659.
91. Aditya K. Sood and Sherali Zeadally. “Malicious AI Models Undermine Software Supply-Chain Security.” Communications of the ACM, 2025.
92. Marcin Spoczynski, Marcela S. Melara, and Sebastian Szyller. “Atlas: A Framework for ML Lifecycle Provenance & Transparency.” EuroS&P Workshops, 2025.
93. Petar Radanliev, Omar Santos, Carsten Maple, and Kayvan Atefi. “Operationalising Artificial Intelligence Bills of Materials for Verifiable AI Provenance and Lifecycle Assurance.” Frontiers in Computer Science, 2026. arXiv:2605.19755.
94. Iain Barclay, Harrison Taylor, Alun D. Preece, Ian J. Taylor, Dinesh C. Verma, and Geeth de Mel. “A Framework for Fostering Transparency in Shared Artificial Intelligence Models by Increasing Visibility of Contributions.” Concurrency and Computation: Practice and Experience, 2021.
95. Iain Barclay, Alun D. Preece, Ian J. Taylor, Swapna Krishnakumar Radha, and Jarek Nabrzyski. “Providing Assurance and Scrutability on Shared Data and Machine Learning Models with Verifiable Credentials.” Concurrency and Computation: Practice and Experience, 2023.
96. Carmine Cesarano and Martin Monperrus. “The Grand Software Supply Chain of AI Systems.” arXiv preprint, 2026. arXiv:2604.27781.
97. Raj Patel, Himanshu Tripathi, Jasper Stone, Noorbakhsh Amiri Golilarz, Sudip Mittal, Shahram Rahimi, and Vini Chaudhary. “Towards Secure MLOps: Surveying Attacks, Mitigation Strategies, and Research Challenges.” arXiv preprint, 2025. arXiv:2506.02032.
98. Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg. “BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain.” arXiv preprint, 2017. arXiv:1708.06733.

99. Yingqi Liu, Shiqing Ma, Yousra Aafer, Wen-Chuan Lee, Juan Zhai, Weihang Wang, and Xiangyu Zhang. “Trojaning Attack on Neural Networks.” NDSS, 2018.
100. Yujie Ji, Xinyang Zhang, Shouling Ji, Xiapu Luo, and Ting Wang. “Model-Reuse Attacks on Deep Learning Systems.” CCS, 2018.
101. Bolun Wang, Yuanshun Yao, Shawn Shan, Huiying Li, Bimal Viswanath, Haitao Zheng, and Ben Y. Zhao. “Neural Cleanse: Identifying and Mitigating Backdoor Attacks in Neural Networks.” IEEE Symposium on Security and Privacy, 2019.
102. Brandon Tran, Jerry Li, and Aleksander Madry. “Spectral Signatures in Backdoor Attacks.” NeurIPS, 2018. arXiv:1811.00636.
103. Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. “Fine-Pruning: Defending Against Backdooring Attacks on Deep Neural Networks.” RAID, 2018. arXiv:1805.12185.
104. Aniruddha Saha, Akshayvarun Subramanya, and Hamed Pirsiavash. “Hidden Trigger Backdoor Attacks.” AAAI, 2020.
105. Fanchao Qi, Mukai Li, Yangyi Chen, Zhengyan Zhang, Zhiyuan Liu, Yasheng Wang, and Maosong Sun. “Hidden Killer: Invisible Textual Backdoor Attacks with Syntactic Trigger.” arXiv preprint, 2021. arXiv:2105.12400.
106. Battista Biggio, Blaine Nelson, and Pavel Laskov. “Poisoning Attacks Against Support Vector Machines.” ICML, 2012.
107. Jacob Steinhardt, Pang Wei Koh, and Percy Liang. “Certified Defenses for Data Poisoning Attacks.” NeurIPS, 2017.
108. Ambra Demontis, Marco Melis, Maura Pintor, Matthew Jagielski, Battista Biggio, Alina Oprea, Cristina Nita-Rotaru, and Fabio Roli. “Why Do Adversarial Attacks Transfer? Explaining Transferability of Evasion and Poisoning Attacks.” USENIX Security, 2019.
109. Antonio Emanuele Cinà, Kathrin Grosse, Ambra Demontis, Battista Biggio, Fabio Roli, and Marcello Pelillo. “Machine Learning Security Against Data Poisoning: Are We There Yet?” arXiv preprint, 2022. arXiv:2204.05986.
110. Sarah Meiklejohn, Hayden Blauzvern, Mihai Maruseac, Spencer Schrock, Laurent Simon, and Ilia Shumailov. “Position: Machine Learning Models Have a Supply Chain Problem.” ICML Position Paper Track, 2025.

1 2 33 <https://dblp.org/pid/27/5136>

<https://dblp.org/pid/27/5136>

3 16 17 <https://arxiv.org/html/2503.04002v1>

<https://arxiv.org/html/2503.04002v1>

4 12 <https://arxiv.org/pdf/2409.09356>

<https://arxiv.org/pdf/2409.09356>

- 5 19 <https://arxiv.org/abs/2306.11673>
<https://arxiv.org/abs/2306.11673>
- 6 10 <https://dblp.org/pid/g/DanielMGerman>
<https://dblp.org/pid/g/DanielMGerman>
- 7 9 31 <https://dblp.org/pid/220/2550.html>
<https://dblp.org/pid/220/2550.html>
- 8 <https://dblp.org/pid/67/3132>
<https://dblp.org/pid/67/3132>
- 11 30 34 <https://dblp.org/db/conf/eurosp/eurosp2025w.html>
<https://dblp.org/db/conf/eurosp/eurosp2025w.html>
- 13 <https://dblp.org/pid/303/7316>
<https://dblp.org/pid/303/7316>
- 14 <https://dblp.org/db/conf/icse/icse2022>
<https://dblp.org/db/conf/icse/icse2022>
- 15 32 <https://dblp.org/pid/53/2641>
<https://dblp.org/pid/53/2641>
- 18 Discovering 1-day Vulnerabilities in Reused C/C++ Open-source ...
https://dblp.org/rec/conf/uss/WooCLO23?utm_source=chatgpt.com
- 20 <https://dblp.org/pid/88/8728>
<https://dblp.org/pid/88/8728>
- 21 <https://dblp.org/pid/94/5094-2>
<https://dblp.org/pid/94/5094-2>
- 22 Dinil Mon Divakaran
https://dblp.org/pid/87/2495?utm_source=chatgpt.com
- 23 Wenxin Jiang 0001
https://dblp.org/pid/71/2402-1?utm_source=chatgpt.com
- 24 Identifying Vulnerabilities in the Machine Learning Model ...
https://dblp.org/rec/journals/corr/abs-1708-06733?utm_source=chatgpt.com
- 25 On the (In)Security of Loading Machine Learning Models
<https://arxiv.org/html/2509.06703v3>
- 26 Towards Secure MLOps: Surveying Attacks, Mitigation Strategies, and Research Challenges
<https://arxiv.org/html/2506.02032v2>
- 27 <https://dblp.org/pid/151/4762>
<https://dblp.org/pid/151/4762>
- 28 <https://dblp.org/pid/74/3780>
<https://dblp.org/pid/74/3780>
- 29 <https://dblp.org/pid/09/486-2>
<https://dblp.org/pid/09/486-2>